Глава 1. Погружаемся и моделируем угрозы!

Любой может научиться моделировать угрозы, и более того — всем следует. Моделирование угроз — это использование моделей для поиска проблем безопасности. Использование модели означает абстрагирование от множества деталей, чтобы взглянуть на более общую картину, а не на сам код. Вы моделируете, потому что это позволяет находить проблемы в том, что ещё не построено, и потому что это позволяет поймать проблему до того, как она начнётся. И наконец, вы моделируете угрозы как способ предвидеть угрозы, которые могут вас затронуть.

Моделирование угроз — прежде всего практическая дисциплина, и эта глава построена так, чтобы отражать эту практичность. Хотя книга даст вам множество ценных определений, теорий, философий, эффективных подходов и хорошо проверенных техник, вы захотите, чтобы они были основаны на опыте. Поэтому эта глава избегает сосредоточения на теории и пока игнорирует variations, вместо этого давая вам шанс учиться на опыте.

Если использовать аналогию: когда вы начинаете играть на инструменте, вам нужно развивать мышцы и осознание, играя на инструменте. Вначале это не будет звучать отлично, временами будет разочаровывающе, но по мере занятий вы обнаружите, что становится легче. Вы начнёте попадать в ноты и в ритм. Аналогично, если вы используете простое четырёхшаговое разбиение того, как моделировать угрозы, которое применяется в частях I–III этой книги, вы начнёте развивать свои мышцы. Вы, вероятно, знаете старую шутку о человеке, который останавливает музыканта на улицах Нью-Йорка и спрашивает: «Как мне попасть в Карнеги-холл?» Ответ, конечно: «Практика, практика, практика». Часть этого включает следование, выполнение упражнений и развитие понимания шагов. По мере этого вы начнёте понимать, как различные задачи и техники, составляющие моделирование угроз, соединяются вместе.

В этой главе вы найдёте недостатки безопасности, которые могут существовать в проекте, чтобы вы могли их устранить. Вы научитесь делать это, рассматривая простое веб-приложение с базой данных на бэкенде. Это даст вам представление о том, что может пойти не так, как это устранить и как проверить свою работу. Попутно вы научитесь играть в Elevation of Privilege — серьёзную игру, разработанную, чтобы помочь вам начать моделировать угрозы. Наконец, вы получите практический опыт построения собственной модели угроз, и глава завершится набором контрольных списков, которые помогут вам начать моделировать угрозы.

Обучение моделированию угроз

Вы начинаете моделирование угроз, сосредотачиваясь на четырёх ключевых вопросах:

1. Что вы создаёте?
2. Что может пойти не так?
3. Что вам следует делать с тем, что может пойти не так?
4. Вы сделали приличную работу по анализу?

Отвечая на эти вопросы, вы начинаете и заканчиваете задачами, знакомыми всем технологам: рисованием на доске и управлением багами. Между ними эта глава представит множество новых техник, которые можно использовать для размышлений об угрозах. Если вы запутаетесь, просто вернитесь к этим четырём вопросам. Всё в этой главе предназначено, чтобы помочь вам ответить на один из этих вопросов.

Сначала вы пройдёте по этим вопросам на примере трёхуровневого веб-приложения, а после прочтения вам следует снова пройти эти шаги с чем-то своим для моделирования угроз. Это может быть ПО, которое вы создаёте или развёртываете, или ПО, которое вы рассматриваете для приобретения. Если вы не уверены, что моделировать, можете использовать одну из примерных систем в этой главе или упражнение из Приложения E «Case Studies».

Во второй раз, проходя эту главу, вам понадобится копия игры для моделирования угроз Elevation of Privilege. В игре используется колода карт, которую можно бесплатно скачать по адресу http://www.microsoft.com/security/sdl/adopt/eop.aspx. Для игровой части следует собрать двух–четырёх друзей или коллег.

Вы начинаете с построения диаграммы — первого из четырёх основных видов деятельности в моделировании угроз, объясняемого в следующем разделе. Остальные три включают поиск угроз, их устранение и проверку работы.

Что вы создаёте?

Диаграммы — хороший способ сообщить, что вы создаёте. Есть много способов диаграммировать ПО, и вы можете начать с диаграммы на доске о том, как данные текут через систему. В этом примере вы работаете с простым веб-приложением с веб-браузером, веб-сервером, некоторой бизнес-логикой и базой данных (см. рисунок 1-1).

Веб-браузер → Веб-сервер → Бизнес-логика → База данных

Рисунок 1-1: Диаграмма на доске

Некоторые люди действительно начнут думать о том, что идёт не так, прямо здесь. Например, откуда вы знаете, что веб-браузер использует именно тот человек, которого вы ожидаете? Что произойдёт, если кто-то изменит данные в базе данных? Нормально ли, что информация перемещается из одного блока в другой без шифрования? Возможно, вы захотите на минуту подумать о некоторых вещах, которые могут пойти не так, потому что такие вопросы могут привести вас к вопросу «это разрешено?»

Вы можете создать ещё лучшую модель того, что создаёте, если немного подумаете о том, «кто что контролирует». Это веб-сайт для всего Интернета или внутренний сайт? База данных находится на месте или у веб-провайдера?

Для этого примера предположим, что вы создаёте интернет-сайт и используете вымышленную систему хранения Acme. (Я бы поставил здесь конкретный продукт, но тогда я ошибся бы в какой-нибудь мелочи, и кто-то, конечно, не вы, зациклился бы на этом и пропустил урок моделирования угроз. Поэтому давайте просто назовём это Acme и представим, что оно работает так, как я говорю. Спасибо! Я знал, что вы поймёте.)

Добавление границ, показывающих, кто что контролирует, — простой способ улучшить диаграмму. Вы можете довольно легко увидеть, что угрозы, пересекающие эти границы, вероятно, важны, и это может быть хорошим местом для начала выявления угроз. Эти границы называются границами доверия, и их следует рисовать везде, где разные люди контролируют разные вещи. Хорошие примеры включают:

- Учётные записи (UID в Unix-системах или SID в Windows)
- Сетевые интерфейсы
- Разные физические компьютеры
- Виртуальные машины
- Организационные границы
- Почти везде, где можно аргументировать разные привилегии

ГРАНИЦА ДОВЕРИЯ ПРОТИВ ПОВЕРХНОСТИ АТАКИ

Близко связанное понятие, с которым вы могли столкнуться, — поверхность атаки. Например, корпус корабля — поверхность атаки для торпеды. Борт корабля представляет большую поверхность атаки для подводной лодки, чем нос того же корабля. У корабля могут быть внутренние «границы доверия», такие как водонепроницаемые переборки или сейф капитана. Система, которая предоставляет много интерфейсов, представляет большую поверхность атаки, чем система, предоставляющая мало API или других интерфейсов. Сетевые межсетевые экраны полезны как границы, потому что они уменьшают поверхность атаки относительно внешнего атакующего. Однако, как и сейф капитана, внутри межсетевого экрана всё ещё есть границы доверия. Граница доверия и поверхность атаки — очень похожие взгляды на одно и то же. Поверхность атаки — это граница доверия и направление, откуда атакующий мог бы начать атаку. Многие люди считают эти термины взаимозаменяемыми. В этой книге обычно используется «граница доверия».

На вашей диаграмме нарисуйте границы доверия как рамки (см. рисунок 1-2), показывая, что внутри каждой, с меткой (например, «корпоративный центр обработки данных») рядом с краем рамки.

Веб-браузер → Веб-сервер
Корпоративный центр обработки данных
Веб-хранилище (внешнее)
Бизнес-логика → База данных

Рисунок 1-2: Границы доверия, добавленные к диаграмме на доске

ПРИМЕЧАНИЕ. Некоторые люди не привыкли играть в игры на работе. Другие подходят к новым играм с опаской, особенно когда эти игры включают длинные сложные инструкции. Elevation of Privilege объясняется всего несколькими строками. Вам стоит попробовать.

По мере того как ваша диаграмма становится больше и сложнее, легко пропустить её часть или запутаться в метках на потоках данных. Поэтому может быть очень полезно

пронумеровать каждый процесс, поток данных и хранилище данных на диаграмме, как показано на рисунке 1-3. (Поскольку каждая граница доверия должна иметь уникальное имя, отражающее уникальное доверие внутри неё, нумеровать их мало смысла.)

Веб-браузер 1 → Веб-сервер 2 3 → Бизнес-логика 4 5 → База данных 6 7
Корпоративный центр обработки данных
Веб-хранилище (внешнее)

Рисунок 1-3: Номера и границы доверия, добавленные к диаграмме на доске

Что касается физической формы диаграммы: используйте то, что работает для вас. Если это диаграмма на доске и фотография с телефона — отлично. Если это Visio, OmniGraffle или другая программа для рисования — отлично. Вам следует рассматривать диаграммы модели угроз как часть процесса разработки, поэтому старайтесь хранить её в системе контроля версий вместе со всем остальным.

Теперь, когда у вас есть диаграмма, естественно спросить: правильная ли это диаграмма? Пока есть простой ответ: предположим, что да. Позже в этой главе есть несколько советов и контрольных списков, а также раздел об обновлении диаграммы, но на данном этапе у вас достаточно хорошая диаграмма, чтобы начать выявлять угрозы, — ради чего вы и купили эту книгу. Итак, давайте выявлять.

Что может пойти не так?

Теперь, когда у вас есть диаграмма, вы действительно можете начать искать, что может пойти не так с её безопасностью. Это настолько весело, что я превратил это в игру под названием Elevation of Privilege. Подробнее об игре — в Приложении D «Elevation of Privilege: The Cards», где обсуждается каждая карта, и в главе 11 «Threat Modeling Tools», где рассматриваются история и философия игры, но вы можете начать играть прямо сейчас по нескольким простым инструкциям. Если вы ещё этого не сделали, скачайте колоду карт по адресу http://www.microsoft.com/security/sdl/adopt/eop.aspx. Распечатайте страницы в цвете и разрежьте их на отдельные карты. Затем перетасуйте колоду и раздайте её друзьям, которых пригласили играть.

Как играть в Elevation of Privilege

Elevation of Privilege — серьёзная игра, разработанная, чтобы помочь вам моделировать угрозы. Пример карты показан на рисунке 1-4. Вы заметите, что, как у игральных карт, у неё есть номер и масть в верхнем левом углу, а пример угрозы — основной текст на карте. Чтобы играть, просто следуйте инструкциям в предстоящем списке.

Tampering
Атакующий может воспользоваться вашим самодельным обменом ключами или контролем целостности, который вы построили вместо использования стандартной криптографии.

Рисунок 1-4: Карта Elevation of Privilege

1. Раздайте колоду. (Тасовка необязательна.)
2. Человек с тройкой Tampering ведёт первый раунд. (В карточных играх, подобных этой, раунды также называются «взятками» или «руками».)

3. Каждый раунд работает так:
   A. Каждый игрок играет одну карту, начиная с ведущего раунд, затем по часовой стрелке.
   B. Чтобы сыграть карту, прочитайте её вслух и попытайтесь определить, влияет ли она на систему, которую вы изобразили на диаграмме. Если можете связать её, запишите это и начислите себе очко. Игра продолжается по часовой стрелке со следующим игроком.
   C. Когда каждый игрок сыграл карту, игрок, сыгравший старшую карту, выигрывает раунд. Этот игрок ведёт следующий раунд.
4. Когда все карты сыграны, игра заканчивается, и побеждает человек с наибольшим количеством очков.
5. Если вы моделируете угрозы для системы, которую создаёте, то вы заводите баги на всё найденное.

Есть люди, которые моделируют угрозы так во сне или даже с трудом могут это отключить. Не все такие. Это нормально. Моделирование угроз — не ракетостроение. Это то, чему может научиться любой, кто участвует в разработке ПО. Не каждый хочет посвящать время тому, чтобы научиться делать это во сне.

Выявление угроз может казаться многим устрашающим. Если вы из таких, не волнуйтесь. Этот раздел предназначен, чтобы мягко провести вас через выявление угроз. Помните, что нужно получать удовольствие. Как сказал один рецензент: «Играть в Elevation of Privilege должно быть весело. Не преуменьшайте это. Мы играем каждую пятницу. Это приятно, расслабляет и всё ещё имеет деловую ценность».

Вне контекста игры вы можете сделать следующий шаг в моделировании угроз, подумав о вещах, которые могут пойти не так. Например, откуда вы знаете, что веб-браузер использует именно тот человек, которого вы ожидаете? Что произойдёт, если кто-то изменит данные в базе данных? Нормально ли, что информация перемещается из одного блока в другой без шифрования? Вам не нужно придумывать эти вопросы, просто глядя на диаграмму и почёсывая подбородок. (Я не придумывал!) Вы можете выявлять подобные угрозы, используя простую мнемонику STRIDE, подробно описанную в следующем разделе.

Использование мнемоники STRIDE для поиска угроз

STRIDE — это мнемоника для того, что идёт не так в безопасности. Она расшифровывается как Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service и Elevation of Privilege:

- **Spoofing** — это притворяться чем-то или кем-то, кем вы не являетесь.
- **Tampering** — это изменение чего-то, что вы не должны изменять. Это может включать пакеты в проводной (или беспроводной) сети, биты на диске или биты в памяти.
- **Repudiation** означает заявление, что вы чего-то не делали (независимо от того, делали вы это или нет).

- **Denial of Service** — это атаки, предназначенные помешать системе предоставлять сервис, включая её крах, доведение до нерабочей медленности или заполнение всего её хранилища.
- **Information Disclosure** — это раскрытие информации людям, не имеющим права её видеть.
- **Elevation of Privilege** — это когда программа или пользователь технически способны делать вещи, которые им не положено делать.

ПРИМЕЧАНИЕ. Отсюда игра Elevation of Privilege получила своё название. В этой книге для игры используется Elevation of Privilege курсивом или сокращение EoP — чтобы избежать путаницы с угрозой.

Вспомните три примерные угрозы, упомянутые в предыдущем разделе:

- Откуда вы знаете, что веб-браузер использует именно тот человек, которого вы ожидаете?
- Что произойдёт, если кто-то изменит данные в базе данных?
- Нормально ли, что информация идёт из одного блока в другой без шифрования?

Это примеры подмены, несанкционированного изменения и раскрытия информации. Использование STRIDE как мнемоники может помочь вам пройти по диаграмме и выбрать примерные угрозы. Соедините это с небольшим знанием безопасности и правильными техниками, и вы будете находить важные угрозы быстрее и надёжнее. Если у вас есть процесс, обеспечивающий разработку модели угроз, документирование её, вы можете повысить уверенность в своём ПО.

Теперь, когда STRIDE в вашем наборе инструментов, пройдите по диаграмме снова и поищите больше угроз, на этот раз используя мнемонику. Составляйте список по ходу: угроза и элемент диаграммы, на который она влияет. (Обычно затрагивается ПО, поток данных или хранилище, а не граница доверия.) Следующий список даёт примеры каждой угрозы.

- **Spoofing**: Кто-то может притвориться другим клиентом, поэтому вам понадобится способ аутентификации пользователей. Кто-то также может притвориться вашим веб-сайтом, поэтому вам следует убедиться, что у вас есть SSL-сертификат и что вы используете единый домен для всех страниц (чтобы помочь тем клиентам, которые читают URL и проверяют, там ли они). Кто-то также может разместить глубокую ссылку на одну из ваших страниц, например logout.html или placeorder.aspx. Вам следует проверять поле Referrer перед выполнением действия. Это не полное решение так называемых CSRF-атак (Cross Site Request Forgery), но начало.
- **Tampering**: Кто-то может изменить данные в вашем бэкенде у Acme. Кто-то может изменить данные, пока они передаются туда и обратно между их центром обработки данных и вашим. Программист может заменить рабочий код на веб-фронтенде без тестирования, думая, что загружает его в staging. Разгневанный

программист может добавить купон «PayBobMore», дающий 20-процентную скидку на все продаваемые товары.

- **Repudiation**: Любое из предыдущих действий может потребовать выяснения, что произошло. Есть ли системные логи? Регистрируется ли нужная информация эффективно? Защищены ли логи от изменения?
- **Information Disclosure**: Что произойдёт, если Acme прочитает вашу базу данных? Может ли кто угодно подключиться к базе данных и читать или записывать информацию?
- **Denial of Service**: Что произойдёт, если тысяча клиентов одновременно придут на сайт? Что, если Acme выйдет из строя?
- **Elevation of Privilege**: Возможно, веб-фронтенд — единственное место, куда должны обращаться клиенты, но что это обеспечивает? Что мешает им подключиться напрямую к серверу бизнес-логики или загрузить новый код? Если есть межсетевой экран, правильно ли он настроен? Что контролирует доступ к вашей базе данных у Acme, или что произойдёт, если сотрудник Acme ошибётся или даже захочет отредактировать ваши файлы?

Предыдущие возможности не предназначены быть полным списком того, как каждая угроза может проявиться против каждой модели. Более полный список можно найти в главе 3 «STRIDE». Эта короткая версия поможет вам начать, и она сосредоточена на том, что вам, возможно, нужно исследовать на основе очень простой диаграммы на рисунке 1-2. Помните аналогию с музыкальным инструментом. Если вы попытаетесь начать играть на фортепиано «Гаспар из тьмы» Равеля (считающийся одним из самых сложных фортепианных произведений когда-либо написанных), вы будете разочарованы.

Советы по выявлению угроз

Независимо от того, выявляете ли вы угрозы с помощью Elevation of Privilege, STRIDE или обоих, вот несколько советов, которые помогут вам оставаться на правильном пути к определению того, что может пойти не так:

- **Начинайте с внешних сущностей**: Если вы не уверены, с чего начать, начните с внешних сущностей или событий, которые движут деятельностью. Есть много других обоснованных подходов: можно начать с веб-браузера, искать подмену, затем несанкционированное изменение и так далее. Можно также начать с бизнес-логики, если, возможно, ведущий разработчик этого компонента в комнате. Где бы вы ни решили начать, стремитесь к некоторому уровню организации. Можно также идти «в порядке STRIDE» по диаграмме. Без некоторой организации трудно понять, когда вы закончили, но будьте осторожны, чтобы не добавить столько структуры, что она подавит креативность.
- **Никогда не игнорируйте угрозу только потому, что сейчас вы ищете не её.** Вы можете придумать некоторые угрозы, рассматривая другие категории. Запишите их и вернитесь к ним позже. Например, вы могли подумать: «может ли кто-нибудь подключиться к нашей базе данных», что указано в разделе раскрытия информации, пока искали угрозы подмены. Если так — отлично! Хорошая работа!

Избыточность в найденном может быть утомительной, но она помогает не пропустить вещи. Если вы поймаете себя на вопросе, является ли «кто-то, не имеющий права подключаться к базе данных, читает информацию» подменой или раскрытием информации, ответ: какая разница? Запишите проблему и двигайтесь к следующей. STRIDE — инструмент, направляющий вас к угрозам, а не требующий категоризировать найденное; как таксономия он всё равно плох. (То есть есть масса проблем безопасности, для которых можно привести аргументы в пользу разных категорий. Сравните и противопоставьте её хорошей таксономии, например таксономии жизни. Есть ли у неё позвоночник? Если да, это позвоночное.)

- **Сосредоточьтесь на осуществимых угрозах**: По ходу вы можете придумать угрозы вроде «кто-то может вставить бэкдор на фабрике чипов» или «кто-то может нанять наш клининговый персонал, чтобы подключить аппаратный кейлоггер и украсть все наши пароли». Это реальные возможности, но не очень вероятные по сравнению с использованием эксплойта для атаки на уязвимость, для которой вы не применили патч, или обманом человека, чтобы он установил ПО. Также возникает вопрос, что вы можете сделать с любым из них, что подводит нас к следующему разделу.

Устранение каждой угрозы

Теперь у вас должен быть приличный по размеру список или списки угроз. Следующий шаг в процессе моделирования угроз — пройти по спискам и устранить каждую угрозу. Существует четыре типа действий, которые можно предпринять против каждой угрозы: смягчить её, устранить её, передать её или принять её. Следующий список кратко рассматривает каждый из этих способов устранения угроз, а затем в последующих разделах вы узнаете, как устранять каждую конкретную угрозу, выявленную с помощью списка STRIDE в разделе «Что может пойти не так». Более подробно о каждой стратегии и техниках устранения этих угроз см. главы 8 и 9 «Defensive Building Blocks» и «Tradeoffs When Addressing Threats».

- **Смягчение угроз** — это действия, чтобы сделать использование угрозы более трудным. Требование паролей для контроля того, кто может войти, смягчает угрозу подмены. Добавление контроля паролей, требующего сложности или срока действия, делает менее вероятным, что пароль будет угадан или использован, если украден.

- **Устранение угроз** почти всегда достигается устранением функций. Если у вас есть угроза, что кто-то получит доступ к административной функции веб-сайта, посетив URL /admin/, вы можете смягчить её паролями или другими техниками аутентификации, но угроза всё ещё присутствует. Вы можете сделать её менее вероятной для обнаружения, используя URL вроде /j8e8vg21euwq/, но угроза всё ещё присутствует. Вы можете устранить её, удалив интерфейс, осуществляя администрирование через командную строку. (Всё ещё есть угрозы, связанные с тем, как люди входят в командную строку. Уход от HTTP делает угрозу легче смягчить, контролируя поверхность атаки. Обе угрозы были бы найдены в полной модели угроз.) Кстати, есть другие способы устранить угрозы, если вы мафиозный

босс или управляете полицейским государством, но я не рекомендую их использование.

- **Передача угроз** — это позволить кому-то или чему-то другому справляться с риском. Например, вы можете передать угрозы аутентификации операционной системе или обеспечение границ доверия продукту межсетевого экрана. Вы также можете передать риск клиентам, например, попросив их пройти через множество труднопонятных диалогов, прежде чем они смогут сделать нужную работу. Очевидно, это не отличное решение, но иногда у людей есть знания, которые они могут внести в компромисс по безопасности. Например, они могут знать, что только что подключились к беспроводной сети кофейни. Если вы считаете, что человек обладает существенными знаниями, которые стоит внести, вам следует поработать, чтобы помочь ей привнести их в решение. Подробнее об этом — в главе 15 «Human Factors and Usability».
- **Принятие риска** — последний подход к устранению угроз. Для большинства организаций большую часть времени досмотр каждого на входе и выходе из здания не стоит расходов или цены для достоинства и удовлетворённости работой этих работников. (Однако алмазные шахты и иногда государственные учреждения придерживаются иного подхода.) Аналогично стоимость предотвращения того, чтобы кто-то вставил бэкдор в материнскую плату, высока, поэтому для каждого из этих примеров вы можете выбрать принятие риска. И как только вы приняли риск, вам не следует о нём беспокоиться. Иногда беспокойство — признак того, что риск не был полностью принят или что принятие риска было неуместным.

Стратегии, перечисленные в следующих таблицах, предназначены служить примерами, иллюстрирующими способы устранения угроз. Вашим «основным» подходом должно быть смягчение угроз. Смягчение, как правило, самое лёгкое и лучшее для ваших клиентов. (Может показаться, что принятие риска легче, но со временем смягчение легче.) Смягчение угроз может быть тяжёлой работой, и не следует считать эти примеры полными. Часто есть другие обоснованные способы устранения каждой из этих угроз, и иногда приходится идти на компромиссы в том, как угрозы устраняются.

Устранение подмены

Таблица 1-1 и следующий за ней список показывают цели подмены, стратегии смягчения, которые устраняют подмену, и техники реализации этих мер.

Таблица 1-1: Устранение угроз подмены

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
Подмена человека	Идентификация и аутентификация (имена пользователей и то, что вы	Имена пользователей, настоящие имена или другие	Регистрация/сопровождение/истечение срока

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
	знаете/имеете/являетесь)	идентификаторы: пароли, токены, биометрия	
Подмена «файла» на диске	Использование ОС	Полные пути, проверка ACL, обеспечение правильного создания каналов	Криптографические аутентификаторы
			Цифровые подписи или аутентификаторы
Подмена сетевого адреса	Криптографическая	DNSSEC, HTTPS/SSL, IPsec	
Подмена программы в памяти	Использование ОС	Многие современные операционные системы имеют некоторую форму идентификатора приложения, которую ОС будет обеспечивать.	

- Когда вы обеспокоены подменой человека, убедитесь, что у каждого человека есть уникальное имя пользователя и какой-то способ аутентификации. Традиционный способ — пароли, у которых есть всевозможные проблемы, а также всевозможные преимущества, которые трудно воспроизвести. См. главу 14 «Accounts and Identity» о паролях.

- При доступе к файлу на диске не запрашивайте файл через open(file). Используйте open(/path/to/file). Если файл чувствительный, после открытия проверьте различные элементы безопасности файлового дескриптора (такие как полностью разрешённое имя, права и владелец). Вы хотите проверять по файловому дескриптору, чтобы избежать состояний гонки. Это особенно верно, когда файл является исполняемым, хотя проверка после открытия может быть сложной. Поэтому может помочь обеспечить, чтобы права на исполняемый файл не могли быть изменены атакующим. В любом случае вы почти никогда не хотите вызывать exec() с ./file.
- Когда вы обеспокоены подменой системы или компьютера при подключении по сети, вы захотите использовать DNSSEC, SSL, IPsec или их комбинацию, чтобы убедиться, что подключаетесь к правильному месту.

Устранение несанкционированного изменения

Таблица 1-2 и следующий за ней список показывают цели несанкционированного изменения, стратегии смягчения, которые устраняют несанкционированное изменение, и техники реализации этих мер.

Таблица 1-2: Устранение угроз несанкционированного изменения

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
Изменение файла	ACL операционной системы	Криптографическая	Цифровые подписи, MAC с ключом
Гонка при создании файла (изменение файловой системы)	Использование каталога, защищённого от произвольного изменения пользователями	ACL	Использование частных структур каталогов (Рандомизация имён файлов лишь делает выполнение атаки более раздражающим.)
Изменение сетевого пакета	Криптографическая	HTTPS/SSL, IPsec	
	Антипаттерн	Изоляция сети (См. примечание об антипаттерне изоляции сети.)	

- **Изменение файла**: Изменение файлов может быть лёгким, если у атакующего есть учётная запись на той же машине, или путём изменения сети, когда файлы получаются с сервера.
- **Изменение памяти**: Угрозы, о которых стоит беспокоиться, — те, которые могут возникнуть, когда процесс с меньшими привилегиями, чем у вас, или которому вы не доверяете, может изменять память. Например, если вы получаете данные из сегмента разделяемой памяти, защищён ли он ACL так, чтобы его видел только другой процесс? Для веб-приложения, куда данные поступают через AJAX, убедитесь, что вы проверяете, что данные соответствуют ожидаемым, после того как извлекли нужный объём.
- **Изменение сетевых данных**: Предотвращение изменения сетевых данных требует решения как подмены, так и изменения. Иначе кто-то, желающий изменить данные, может просто притвориться другим концом, используя так называемую атаку «человек посередине». Самое распространённое решение этих проблем — SSL, а IP Security (IPsec) появляется как другая возможность. SSL и IPsec оба решают конфиденциальность и изменение и могут помочь решить подмену.

Устранение репудиации

Устранение репудиации, как правило, сводится к обеспечению того, чтобы система была спроектирована для ведения логов, и чтобы эти логи сохранялись и защищались. Часть этого можно решить простыми шагами, например использованием надёжного транспорта для логов. В этом смысле syslog over UDP почти всегда был глупым с точки зрения безопасности; syslog over TCP/SSL теперь доступен и значительно лучше.

Таблица 1-3: Устранение угроз репудиации

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
Нет логов — ничего не докажешь.	Log	Обязательно логируйте всю информацию, относящуюся к безопасности.	
Логи подвергаются атаке	Защитите свои логи.	Отправка по сети. ACL	
Логи как канал для атаки	Строго определённые логи	Документирование дизайна логов на раннем этапе разработки	

- **Нет логов — ничего не докажешь**: Это самоочевидно. Например, когда клиент звонит с жалобой, что он не получил заказ, как это будет разрешено? Ведите логи,

чтобы можно было расследовать, что происходит, когда кто-то пытается что-то отрицать.

- **Логи подвергаются атаке**: Атакующие будут делать вещи, чтобы ваши логи не были полезны, включая заполнение лога, чтобы было трудно найти атаку, или заставляя логи «перекатываться». Они также могут делать вещи, чтобы вызвать так много тревог, что реальная атака потеряется в море проблем. Возможно, очевидно, что отправка логов по сети подвергает их другим угрозам, с которыми вам придётся разбираться.
- **Логи как канал для атаки**: По замыслу вы собираете данные из источников вне вашего контроля и доставляете эти данные людям и системам с привилегиями безопасности. Пример такой атаки — отправка почты на адрес «</html> haha@example.com», что создаёт проблемы для веб-инструментов, не ожидающих встроенного HTML.
  Вы можете упростить написание безопасного кода для обработки логов, чётко сообщая, что ваши логи не могут содержать, например: «Наши логи все текстовые, и атакующие могут вставлять всевозможные вещи», или «Поля 1–5 наших логов строго контролируются нашим ПО, поля 6–9 легко поддаются инъекции данных. Поле 1 — время в GMT. Поля 2 и 3 — IP-адреса (v4 или 6)...» Если у вас нет невероятно строгого контроля, документирование того, что могут содержать ваши логи, вероятно, что-то упустит. (Например, могут ли ваши логи содержать Unicode-символы двойной ширины?)

Устранение раскрытия информации

Таблица 1-4 и следующий за ней список показывают цели раскрытия информации, стратегии смягчения, которые устраняют раскрытие информации, и техники реализации этих мер.

Таблица 1-4: Устранение угроз раскрытия информации

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
Мониторинг сети	Шифрование	HTTPS/SSL, IPsec	
Имя каталога или файла (например, layoff-letters/adamshostack.docx)	Использование ОС.	ACL	
Содержимое файла	Использование ОС.	ACL, криптография	Шифрование файлов, такое как PGP, шифрование

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
			диска (FileVault, BitLocker)
Раскрытие информации через API	Проектирование	Тщательный контроль дизайна. Рассмотрите передачу по ссылке или по значению.	

- **Мониторинг сети**: Мониторинг сети использует архитектуру большинства сетей для наблюдения за трафиком. (В частности, большинство сетей теперь широковещательно рассылают пакеты, и каждый слушатель должен решить, важен ли для него пакет.) Когда сети устроены иначе, есть множество техник, чтобы привлечь трафик к станции мониторинга или через неё.
  Если вы не устраняете подмену, как и в случае с изменением, атакующий может просто сесть в середину и подменить каждый конец. Смягчение угроз раскрытия сетевой информации требует решения как угроз подмены, так и изменения. Если вы не устраняете изменение, то есть всевозможные умные способы вытащить информацию. Здесь, опять же, SSL и IP Security — ваши простейшие варианты.
- **Имена раскрывают информацию**: Когда имя каталога или имя файла само по себе раскрывает информацию, лучший способ защитить его — создать родительский каталог с безобидным именем и использовать ACL или права операционной системы.
- **Содержимое файла чувствительно**: Когда содержимое файла нужно защитить, используйте ACL или криптографию. Если вы хотите защитить все данные, если машина попадёт в несанкционированные руки, вам понадобится криптография. Формы криптографии, требующие ручного ввода ключа или парольной фразы, более безопасны и менее удобны. Существует шифрование файлов, файловых систем и баз данных — в зависимости от того, что вам нужно защитить.
- **API раскрывают информацию**: При проектировании API или иной передаче информации через границу доверия тщательно выбирайте, какую информацию раскрывать. Следует исходить из того, что предоставленная вами информация будет передана другим, поэтому будьте избирательны. Например, ошибки веб-сайта, раскрывающие имя пользователя и пароль к базе данных, — распространённая форма этого недостатка; другие обсуждаются в главе 3.

Устранение отказа в обслуживании

Таблица 1-5 и следующий за ней список показывают цели отказа в обслуживании, стратегии смягчения, которые устраняют отказ в обслуживании, и техники реализации этих мер.

Таблица 1-5: Устранение угроз отказа в обслуживании

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
Переполнение сети	Ищите исчерпаемые ресурсы.	Эластичные ресурсы. Работайте над тем, чтобы потребление ресурсов атакующим было таким же или выше вашего.	Сетевые ACL
Ресурсы программы	Тщательный дизайн	Эластичное управление ресурсами, доказательство работы. Избегайте множителей. Ищите места, где атакующие могут умножить потребление CPU на вашей стороне с минимальными усилиями на своей: сделайте что-то, что требует работы или позволяет различать атакующих, например клиент сначала делает криптографию или вход перед большими вычислительными затратами (конечно, это не должно означать, что входы не шифруются).	
Системные ресурсы	Использование ОС.	Используйте настройки ОС.	

- **Переполнение сети**: Если у вас есть статические структуры для числа соединений, что произойдёт, если они заполнятся? Аналогично, в той мере, в какой это под вашим контролем, не принимайте небольшой объём сетевых данных с возможно подменённого адреса и не возвращайте много данных. Наконец, межсетевые экраны могут обеспечить уровень сетевых ACL для контроля того, откуда вы будете принимать (или куда отправлять) трафик, и могут быть полезны для смягчения сетевых атак отказа в обслуживании.

- **Ищите исчерпаемые ресурсы**: Первый набор исчерпаемых ресурсов связан с сетью, второй — те, которыми управляет ваш код, третий — те, которыми управляет ОС. В каждом случае эластичное выделение ресурсов — ценная техника. Например, в 1990-х некоторые стеки TCP имели жёсткое ограничение в пять полуоткрытых TCP-соединений. (Полуоткрытое соединение — это соединение в процессе открытия. Не волнуйтесь, если это не имеет смысла, но лучше спросите себя, почему код был ограничен пятью.) Сегодня вы часто можете получить эластичное выделение различных ресурсов у облачных провайдеров.
- **Системные ресурсы**: Операционные системы, как правило, имеют лимиты или квоты для контроля потребления ресурсов пользовательским кодом. Рассмотрите те ресурсы, которыми управляет операционная система, такие как память или использование диска. Если ваш код работает на выделенных серверах, может быть разумно позволить ему съедать всю машину. Будьте осторожны, если снимаете ограничения с кода, и обязательно документируйте то, что делаете.
- **Ресурсы программы**: Рассмотрите ресурсы, которыми управляет сама ваша программа. Также подумайте, может ли атакующий заставить вас делать больше работы, чем делает он. Например, если он отправляет вам пакет, полный случайных данных, а вы выполняете над ним дорогие криптографические операции, то ваша уязвимость к отказу в обслуживании будет выше, чем если вы заставите его сначала выполнить криптографию. Конечно, в эпоху ботнетов есть пределы тому, насколько хорошо можно перераспределить эту работу. Есть отличная статья Бена Лори и Ричарда Клейтона «Proof of work proves not to work», которая выступает против схем доказательства работы (Laurie, 2004).

Устранение повышения привилегий

Таблица 1-6 и следующий за ней список показывают цели повышения привилегий, стратегии смягчения, которые устраняют повышение привилегий, и техники реализации этих мер.

Таблица 1-6: Устранение угроз повышения привилегий

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
Путаница данных/кода	Используйте инструменты и архитектуры, разделяющие данные и код.	Подготовленные выражения или хранимые процедуры в SQL. Чёткие разделители с каноническими формами. Поздняя валидация того, что данные	

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
		соответствуют ожиданиям следующей функции.	
Атаки на поток управления/повреждение памяти	Используйте типобезопасный язык.	Написание кода на типобезопасном языке защищает от целых классов атак.	
	Использование ОС для защиты памяти.	Большинство современных операционных систем имеют средства защиты памяти.	
	Используйте песочницу.	Современные операционные системы поддерживают песочницу разными способами (AppArmor в Linux, AppContainer или шаблон MOICE в Windows, Sandboxlib в Mac OS). Не запускайте под учётной записью «nobody», создайте новую для каждого приложения. Postfix и QMail — примеры хорошего шаблона «одна учётная	

УГРОЗА	ЦЕЛЬ	СТРАТЕГИЯ СМЯГЧЕНИЯ	ТЕХНИКА СМЯГЧЕНИЯ
		запись на функцию».	
Атаки с инъекцией команд	Будьте осторожны.	Проверяйте, что ваш ввод имеет ожидаемый размер и форму. Не «санитизируйте». Логируйте и затем отбрасывайте, если он странный.	

- **Путаница данных/кода**: Проблемы, когда данные трактуются как код, распространены. Когда информация пересекает слои, то, что заражено, а что чисто, может теряться. Атаки вроде XSS пользуются тем, что HTML свободно переплетает код и данные. (То есть .html-файл содержит и код, например JavaScript, и данные, например текст для отображения, а иногда и инструкции форматирования для этого текста.) Есть несколько стратегий работы с этим. Первая — искать способы, которыми фреймворки помогают разделять код и данные. Например, подготовленные выражения в SQL говорят базе данных, какие выражения ожидать и где будут данные.
  Вы также можете посмотреть на данные, которые передаёте, прямо перед передачей, чтобы знать, какую валидацию от вас может ожидать вызываемая функция. Например, если вы отправляете данные на веб-страницу, вы можете убедиться, что они не содержат символов <, >, # или &. На самом деле значение «что угодно» сильно зависит от того, что именно находится между «вами» и отображением веб-страницы, и какие проверки безопасности оно может выполнять. Если «вы» — это веб-сервер, может быть очень важно иметь несколько символов < и > в том, что вы создаёте. Если «вы» — это что-то, берущее данные из базы данных и отправляющее их, скажем, в PHP, то история совсем иная. В идеале природа «вас» и дополнительные шаги ясны на ваших диаграммах.
- **Атаки на поток управления/повреждение памяти**: Этот набор атак обычно использует слабую типизацию и статические структуры в C-подобных языках, чтобы позволить атакующему предоставить код и затем перейти к этому коду. Если вы пишете на типобезопасном языке, вы защищены от целых классов атак. Если вы пишете на C или C++, используйте средства защиты памяти ОС, песочницы и тщательную проверку ввода. Цель — не дать атакующему изменить поток управления или выполнить свой код.

Заводите баги

Теперь, когда у вас есть список угроз и способы, которыми вы хотели бы их смягчить, вы прошли сложные, ориентированные на безопасность части процесса. Осталось сделать ещё несколько вещей, первая из которых — рассматривать каждую строку предыдущих таблиц как баг. Вы хотите рассматривать их как баги, потому что если вы выпускаете ПО, вы научились как-то обрабатывать баги. У вас, вероятно, есть способ отслеживать их, приоритезировать и обеспечивать их закрытие с подходящей степенью согласованности. Это будет означать очень разное для стартапа из трёх человек и для производителя медицинского оборудования, но обе организации будут иметь способ обработки багов. Вы хотите использовать эту процедуру, чтобы моделирование угроз не было просто бумажным упражнением.

Вы можете написать текст багов разными способами, в зависимости от того, что делает ваша организация. Примеры заведения бага могут включать:

- Кто-то может использовать интерфейс /admin/ без надлежащей авторизации.
- Интерфейсу администратора не хватает надлежащего контроля авторизации.
- Нет автоматизированного тестирования безопасности для интерфейса /admin/.

Какой бы способ вы ни выбрали, отлично, если вы можете включить всю угрозу в баг и пометить его как баг безопасности, если ваш инструмент отслеживания багов это поддерживает. (Если вы супер-гибкая scrum-команда, используйте стикер уникального цвета для багов безопасности.) Вам также придётся приоритезировать баги. Баги повышения привилегий почти всегда будут попадать в категорию наивысшего приоритета, потому что при их эксплуатации наносится так много ущерба. Отказ в обслуживании часто падает вниз стека, но вам придётся рассмотреть каждый баг, чтобы определить, как его ранжировать.

Проверка вашей работы

Валидация вашей модели угроз — последнее, что вы делаете как часть моделирования угроз. Здесь нужно выполнить несколько задач, и лучше всего держать их в том же порядке, в котором вы делали предыдущую работу. Поэтому задачи валидации включают проверку модели, проверку того, что вы искали каждую угрозу, и проверку ваших тестов. Вероятно, вы также захотите провести валидацию модели второй раз, когда приблизитесь к выпуску или развёртыванию.

Проверка модели

Вы должны убедиться, что итоговая модель соответствует тому, что вы построили. Если нет, как вы можете знать, что нашли правильные, релевантные угрозы? Для этого попробуйте организовать встречу, на которой все смотрят на диаграмму и отвечают на следующие вопросы:

- Это полно?
- Это точно?
- Это охватывает все решения по безопасности, которые мы приняли?
- Могу ли я начать следующую версию с этой диаграммой без изменений?

Если все говорят «да», ваша диаграмма достаточно актуальна для следующего шага. Если нет, вам нужно её обновить.

Обновление диаграммы

По ходу работы с диаграммой вы могли заметить, что в ней отсутствуют ключевые данные. Если бы это была реальная система, могли бы быть дополнительные интерфейсы, которые не нарисованы, или дополнительные базы данных. Могли быть детали, которые вы подскочили к доске, чтобы нарисовать. Если так, вам нужно обновить диаграмму этими деталями. Несколько правил полезны при создании или обновлении диаграмм:

- Сосредоточьтесь на потоке данных, а не на потоке управления.
- Каждый раз, когда вам нужно оговорить ответ словами «иногда» или «также», рассмотрите добавление更多 деталей, чтобы разбить различные случаи. Например, если вы говорите: «Иногда мы подключаемся к этому веб-сервису через SSL, а иногда откатываемся к HTTP», вам следует нарисовать оба этих потока данных (и рассмотреть, может ли атакующий заставить вас так откатиться).
- Каждый раз, когда вам нужно больше деталей, чтобы объяснить поведение, относящееся к безопасности, рисуйте их.
- В любом месте, где вы спорили о дизайне или конструкции системы, впишите согласованные факты. Это важный шаг, чтобы убедиться, что все закончили обсуждение на одной странице. Это особенно важно для больших команд, когда не все находятся в комнате для обсуждения модели угроз. Если они видят диаграмму, которая противоречит их мышлению, они могут либо принять её, либо оспорить предположения; но в любом случае хорошая чёткая диаграмма может помочь всем прийти к общему пониманию.
- Не имейте стоков данных: вы пишете данные по какой-то причине. Покажите, кто их использует.
- Данные не могут перемещаться сами из одного хранилища в другое: покажите процесс, который их перемещает.
- Диаграмма должна рассказывать историю и поддерживать вас, когда вы рассказываете истории, указывая на неё.
- Не рисуйте глазную диаграмму (диаграмму с таким количеством деталей, что нужно прищуриться, чтобы прочитать мелкий шрифт).

Детали диаграммы

Если вы задаётесь вопросом, как примирить это последнее правило с деталями, которые может нести реальный программный проект, одна из техник — использовать поддиаграмму, показывающую детали одной конкретной области. Вам следует искать способы выделить детали, которые имеют смысл для вашего проекта. Например, если у вас есть один гиперсложный процесс, может быть, всё внутри него — одна диаграмма, а всё снаружи — другая. Если у вас есть диспетчер или система очередей, это хорошее место для разбиения. Ваши базы данных или система отказоустойчивости — тоже

хорошее место для разделения. Может быть, есть набор элементов, которые действительно нуждаются в большей детализации. Всё это — хорошие способы выделить детали.

Главное помнить: диаграмма предназначена, чтобы помочь вам понять и обсудить систему. Вспомните цитату, открывающую книгу: «Все модели неверны. Некоторые модели полезны». Поэтому, когда вы добавляете дополнительные диаграммы, не спрашивайте: «Это правильный способ делать это?» Вместо этого спросите: «Это помогает нам подумать о том, что может пойти не так?»

Проверка каждой угрозы

Есть два основных типа валидационных действий, которые вам следует сделать. Первый — проверить, что вы сделали правильную вещь с каждой найденной угрозой. Другой — спросить, нашли ли вы все угрозы, которые должны были найти.

Что касается проверки, что вы сделали правильную вещь с каждой найденной угрозой, первый и главный вопрос: «Сделал ли я что-то с каждой уникальной угрозой, которую нашёл?» Вы действительно не хотите уронить что-то на пол. Это работа вроде «кручения ручки». Она редко бывает гламурной или захватывающей, пока вы не найдёте то, что упустили. Вы можете сэкономить много времени, ведя протокол встречи и записывая номер бага рядом с каждой угрозой, проверяя, что вы рассмотрели каждую при триаже багов.

Следующий вопрос: «Сделал ли я правильную вещь с каждой угрозой?» Если вы завели баги с каким-то тегом безопасности, выполните запрос по всем багам безопасности и просмотрите каждый. Это может быть так же легко, как прочитать каждый баг и спросить себя: «Сделал ли я правильную вещь?» — или вы можете использовать короткий контрольный список, пример которого («Validating threats») включён в конце этой главы в разделе «Checklists for Diving in and Threat Modeling».

Проверка ваших тестов

Для каждой угрозы, которую вы устраняете, убедитесь, что вы построили хороший тест для обнаружения проблемы. Ваш тест может быть ручным процессом тестирования или автоматизированным тестом. Некоторые из этих тестов будут лёгкими, другие — очень сложными. Например, если вы хотите убедиться, что ни одна статическая веб-страница под /beta не может быть доступна без beta-cookie, вы можете построить быстрый скрипт, который извлекает все страницы из вашего репозитория исходного кода, строит для неё URL и пытается получить страницу. Вы можете расширить скрипт, чтобы отправлять cookie с каждым запросом, а затем повторно запросить с admin-cookie. В идеале это легко сделать в вашем существующем фреймворке веб-тестирования. Это становится немного сложнее с динамическими страницами и намного сложнее, когда риск безопасности — это что-то вроде SQL-инъекции или безопасного парсинга пользовательского ввода. Об этих предметах написаны целые книги, не говоря уже о целых книгах по тестированию. Ключевой вопрос, который вы должны задать: «Соответствуют ли мои тесты безопасности другим программным тестам и типам рисков, которые выявляют сбои?»

Моделирование угроз самостоятельно

Теперь вы прошли через свою первую модель угроз. Поздравляю! Помните, однако: вы не попадёте в Карнеги-холл, если не будете практиковаться, практиковаться и практиковаться. Это значит, что пора сделать это снова, на этот раз самостоятельно, потому что повторение — единственный способ стать лучше. Выберите систему, над которой работаете, и смоделируйте для неё угрозы. Следуйте этому упрощённому пятишаговому процессу:

1. Нарисуйте диаграмму.
2. Используйте игру EoP, чтобы найти угрозы.
3. Устраните каждую угрозу каким-либо способом.
4. Проверьте свою работу с помощью контрольных списков в конце этой главы.
5. Отпразднуйте и поделитесь своей работой.

Прямо сейчас, если вы новичок в моделировании угроз, лучше всего делать это часто, применяя к ПО и системам, которые для вас важны. После моделирования нескольких систем вы обнаружите, что становитесь более комфортными с инструментами и техниками. Пока что нужно практиковаться. Постройте свои первые мышцы для моделирования угроз.

Это поднимает вопрос: что моделировать дальше?

То, над чем вы работаете сейчас, — первое место для поиска следующей системы для моделирования угроз. Если у неё есть какая-то граница доверия, она может быть хорошим кандидатом. Если она слишком проста, чтобы иметь границы доверия, моделирование угроз, вероятно, не будет очень удовлетворительным. Если у неё слишком много границ, это может быть слишком большой проект, чтобы прожевать его сразу. Если вы тесно сотрудничаете с несколькими людьми, которым доверяете, это может быть хорошей возможностью сыграть в EoP с ними. Если вы работаете в большой команде, или через организационные границы, или атмосфера напряжённая, то эти люди могут не быть хорошими первыми collaborators по моделированию угроз. Начните с того, над чем работаете сейчас, если нет ощутимых причин ждать.

Контрольные списки для погружения и моделирования угроз

В этой главе много всего. Когда вы садитесь действительно делать работу самостоятельно, может быть сложно оценить, как вы справляетесь. Вот несколько контрольных списков, предназначенных помочь вам избежать наиболее распространённых проблем. Каждый вопрос предназначен для чтения вслух и утвердительного ответа от всех присутствующих. После чтения каждого вопроса вслух поощряйте вопросы или уточнения от всех остальных участников.

Диаграммирование

1. Можем ли мы рассказать историю, не меняя диаграмму?
2. Можем ли мы рассказать эту историю, не используя слова «иногда» или «также»?
3. Можем ли мы посмотреть на диаграмму и точно увидеть, где ПО будет принимать решение по безопасности?

4. Показывает ли диаграмма все границы доверия, например, где взаимодействуют разные учётные записи? Охватываете ли вы все UID, все роли приложений и все сетевые интерфейсы?
5. Отражает ли диаграмма текущую или планируемую реальность ПО?
6. Видим ли мы, куда идут все данные и кто их использует?
7. Видим ли мы процессы, которые перемещают данные из одного хранилища в другое?

Угрозы

1. Искали ли мы каждую из угроз STRIDE?
2. Посмотрели ли мы на каждый элемент диаграммы?
3. Посмотрели ли мы на каждый поток данных на диаграмме?

ПРИМЕЧАНИЕ. Потоки данных — это тип элемента, но их иногда упускают, когда люди начинают, поэтому вопрос 3 — это вопрос «ремень и подтяжки», добавляющий избыточность. (Подход «ремень и подтяжки» гарантирует, что брюки джентльмена останутся на месте.)

Валидация угроз

1. Записали ли мы или завели баг для каждой угрозы?
2. Есть ли предложенный/запланированный/реализованный способ устранения каждой угрозы?
3. Есть ли у нас тест-кейс на каждую угрозу?
4. Прошло ли ПО тест?

Резюме

Любой технический специалист может научиться моделировать угрозы. Моделирование угроз включает пересечение двух моделей: модели того, что может пойти не так (угрозы), применённой к модели ПО, которое вы создаёте или развёртываете, закодированной в диаграмме. Одна модель угроз — STRIDE: spoofing, tampering, repudiation, information disclosure, denial of service и elevation of privilege. Эта модель угроз была превращена в игру Elevation of Privilege, которая добавляет структуру и подсказки к модели.

С диаграммой на доске и копией Elevation of Privilege разработчики могут моделировать угрозы для ПО, которое они создают; системные администраторы могут моделировать угрозы для ПО, которое они развёртывают, или системы, которую они строят; а специалисты по безопасности могут познакомить с моделированием угроз тех, чьи навыки лежат вне безопасности.

Важно устранять угрозы, и угрозы STRIDE — это инверсия свойств, которые вы хотите. Существуют стратегии и техники смягчения для разработчиков и для системных администраторов.

Как только вы создали модель угроз, важно проверить свою работу, убедившись, что у вас есть хорошая модель ПО в актуальной диаграмме и что вы проверили каждую найденную угрозу.